

ACA ANSWERS

Farina Filippo

1 Define the main characteristics of a VLIW architecture. Explain which is the basic idea behind Trace Scheduling for VLIW and why is it necessary to get good performance from a VLIW?

VLIW (Very Long Instruction Word) architecture is a design approach that shifts the complexity of instruction-level parallelism (ILP) from the hardware to the compiler through static scheduling.

The defining features of a VLIW processor include:

- **Static Scheduling:** Unlike superscalar processors that use hardware for dynamic scheduling, VLIW relies entirely on the compiler to identify parallelism,.
- **Instruction Bundles:** The processor fetches a "bundle" or wide template containing a fixed number of operations (typically 4 to 16) that are executed simultaneously,.
- **Dependency-Free Code:** By construction, the compiler must provide code that is free of data, structural, and control hazards.
This means the hardware does not need to perform complex runtime checks or interlocks.
- **Hardware Simplicity:** Because scheduling decisions are made at compile-time, the control unit is significantly simpler compared to dynamic architectures,.
- **Constant Latencies:** Operations have fixed execution times (e.g., 1 cycle for integers, 3 for memory, and 4 for floating-point operations), which the compiler uses to plan the schedule,.
- **High Resource Demands:** To keep all functional units (FUs) active, VLIW machines require a large number of registers, high data transport capacity (buses), and high bandwidth between the cache and the fetch unit,.

Trace Scheduling is a global code scheduling technique used by compilers to optimize execution beyond the limits of a single basic block.

- **Trace Identification:** A "trace" is defined as a loop-free sequence of basic blocks within the control flow graph that represents a likely execution path based on specific inputs,.
- **Path Selection:** The compiler uses profiling feedback or heuristics to identify the most frequently executed branch paths.
- **Unified Scheduling:** Once a trace is chosen, the compiler schedules the entire sequence of operations as if it were one large basic block, ignoring branches during the initial optimization phase.
- **Compensation Code:** Since moving instructions across branches can affect correctness, the compiler generates compensation code at "side entry" and "side exit" points. This ensures that if the program takes a different path than the one predicted, the results remain correct,.

Why Trace Scheduling is Necessary for Performance

Trace scheduling is vital for VLIW performance for several reasons:

-
- **Limited Basic Block ILP:** The amount of parallelism available within a single basic block (a sequence with no branches) is very small, . Without looking across multiple blocks, a VLIW processor cannot find enough independent instructions to fill its wide word.
- **Maximizing Functional Unit (FU) Utilization:** VLIW processors have many functional units. If the compiler cannot find enough parallel operations, it must insert NOPs (No Operations), which leads to code explosion and poor efficiency, .
- **Hiding Latencies:** Since operations like floating-point additions take multiple cycles, the compiler needs a large window of instructions to "fill the gaps" and keep the pipeline full, .

- **Global Optimization:** It allows the compiler to move instructions from less frequent paths to fill empty slots in the most frequent paths, significantly improving the average execution speed,.

In the absence of global scheduling such as trace scheduling, a VLIW architecture would be unable to exploit its static parallelism, as the lack of dynamic reordering would make the use of functional units highly inefficient

2 Dynamic Branch Predictor : The obtained result, in terms of miss predictions, is inline with theoretical characteristics of the two predictors? Please effectively support your answer

The 2 BHT improves average performance, but it doesn't guarantee an improvement for every single branch:

In a simple and not-nested loop (such as a loop executed only once, like a not nested for loop) the 2-BHT doesn't outperform 1-BHT, they are identification , or in some cases the 2 BHT is worse, in terms of mispredictions.

3 Explain the main characteristics of static scheduling and its implication. Also discuss the trace scheduling and software pipelining techniques.

Static scheduling is a strategy for supporting Instruction-Level Parallelism (ILP) that relies entirely on software—specifically the compiler—to identify, organize, and extract the parallelism present in a program. All decisions regarding the order and parallel execution of instructions are made at compile time and remain unchanged during execution.

The fundamental characteristics of this approach are:

- **Compile-time decisions:** Unlike dynamic scheduling, the instruction order and dependency management are fixed before execution and are not adapted at runtime.
- **Central role of the compiler:** The compiler analyzes the code, reorders it, and produces an output free of dependencies (structural, data, and control hazards), avoiding conflicts during execution.
- **Fixed latencies:** The compiler operates assuming known and constant execution times for each type of operation (e.g., 1 cycle for integer operations, multiple cycles for floating-point).

This approach leads to several architectural and performance consequences:

- **Hardware simplicity:** Since dependency resolution is offloaded to the compiler, the control unit is much simpler compared to systems with dynamic scheduling (such as Tomasulo's algorithm), as it does not have to manage conflicts at runtime.
- **Energy efficiency:** The lower complexity of the hardware, particularly the Control Unit, generally leads to reduced energy consumption.
- **Code explosion:** If the compiler cannot fill all the slots of a VLIW instruction, it must insert NOPs (No-Operations), significantly increasing the code size.
- **Resource requirements:** Many registers are needed to keep functional units active simultaneously, along with high bandwidth between the cache and the fetch unit to transfer very long instructions.

VLIW (Very Long Instruction Word) Architectures: Static scheduling is the foundation of VLIW architectures, where multiple independent operations are grouped into a single very long instruction (bundle) with fixed slots; the hardware simply executes in parallel what the compiler has already planned.

Software pipelining is a static scheduling technique used specifically to optimize the execution of loops.

- **Core Concept:** Instead of executing one complete iteration after another, the compiler reorganizes the code to overlap the execution of successive iterations. Consequently, parts of different iterations can be executed in a single clock cycle.
- **Code Structure:** The resulting code is divided into three parts:
 1. Prolog: The initial startup phase that fills the software pipeline.
 2. Loop Body / Kernel: The central part where the pipeline is at full capacity and maximum parallelism is achieved.
 3. Epilog: The final phase that completes the remaining pending operations.
- **Performance Benefits:** The cost of starting and closing the pipeline is paid only once for the entire loop rather than for every iteration, allowing for efficient use of functional units and, ideally, a CPI (Cycles Per Instruction) close to 1.

Trace scheduling is a global code scheduling technique designed to extract parallelism beyond individual **basic blocks**. It is particularly useful when loops are absent or when basic blocks offer little parallelism.

- **Definition of a Trace:** A trace is a sequence of loop-free basic blocks that represents a likely execution path in the program's control flow graph.
- **Path Identification:** The compiler uses profiling or heuristics to identify the most frequently executed branches and paths.
- **Scheduling Mechanism:** The entire trace is scheduled as if it were one large basic block, maximizing parallelism along the most common path.
- **Compensation Code:** Since the trace is based on a prediction, the compiler must insert compensation code at side entry and side exit points. This ensures data correctness and exception handling whenever the actual execution flow deviates from the predicted path.
- **Branch Management:** To mitigate the impact of branches, predicated instructions or speculative instructions can be used, further increasing opportunities for parallelism.

4 Describe the Flynn's taxonomy providing for each class the main features and real examples

Flynn's Taxonomy is a classification of parallel computer architectures based on the combination of instruction streams and data streams. The four primary classes are described below:

- **SISD (Single Instruction, Single Data).**

Characteristics: Represents the traditional serial (non-parallel) computer. In each clock cycle, the CPU processes a single instruction stream that operates on a single data stream. Execution is deterministic.

Real-world Examples: Uniprocessor systems and older standard microprocessors.

- **SIMD (Single Instruction, Multiple Data)**

Characteristics: A single control unit (one Program Counter) broadcasts the same instruction to multiple processing elements.

Each processing unit executes the same instruction during the same clock cycle, but on different data elements.

This model is characterized by low latency, high throughput but limited flexibility, and a simple programming model.

Real-world Examples: Vector processors, multimedia instructions (like SSE/AVX), graphics/image processing, and custom integrated circuits. A specific example is the SPU (Synergistic Processing Unit) in the Cell processor.

- **MISD (Multiple Instruction, Single Data)**

Characteristics: In this configuration, a single data stream is sent to multiple processing units. Each unit operates on the data independently through different instruction streams.

Real-world Examples: There are no practical configurations or commercial systems based on this class. It is mostly a theoretical category

- **MIMD (Multiple Instruction, Multiple Data)**

Characteristics: This is the most common type of modern parallel computer. Each processor has its own Program Counter and can execute different instructions on different data. Execution can be synchronous or asynchronous, deterministic or non-deterministic. These systems are scalable, fault-tolerant, and often utilize standard microprocessors.

Real-world Examples: Multi-core processors, shared-memory multiprocessor systems (SMP/UMA or NUMA) and High-Performance Computing (HPC) clusters.

5 Which problem solve the branch prediction? Please list a technique for each of the two main classes of branch prediction and describe it briefly

Branch prediction solves the problem of Control Hazards within a pipeline. These hazards arise when the processor must decide which instruction to fetch before the result of a conditional branch has actually been calculated.

Without a prediction technique, the pipeline would have to stall while waiting for the branch to be resolved, drastically reducing system performance and throughput.

The two main classes of branch prediction are **static** and **dynamic**.

A technique for each class is described below:

- **Static Technique: Branch Always Not Taken:**

In this technique, the actions for a branch are fixed at compile or design time and do not change during execution.

The processor assumes the branch will never be taken, continuing to fetch instructions sequentially.

If the branch condition is indeed not met, the pipeline proceeds without interruption, preserving performance.

If, however, the branch must be taken, the incorrectly fetched instructions are **discarded (flushed)**, and execution restarts from the branch's target address, incurring a performance penalty proportional to the pipeline depth

- **Dynamic Technique: 1-bit Branch History Table**

In this class, the prediction decision can change during program execution based on behavior observed at runtime.

This technique utilizes a table (called the Branch History Table or BHT) indexed by the lower bits of the branch instruction address. Each entry in the table contains 1 bit that indicates whether the branch was taken (1) or not (0) the last time it was encountered. The processor uses this bit as a "hint" for future predictions: if the prediction proves incorrect, the bit is flipped and stored again to correct the behavior for subsequent executions.

6 Superscalar

6.1 Superscalar processors today use an approach that exploits both Instruction Level Parallelism and Thread Level Parallelism. Briefly explain the approach

The approach used by modern superscalar processors to combine Instruction Level Parallelism (ILP) and Thread Level Parallelism (TLP) is known as **Simultaneous Multithreading (SMT)**

By combining these two forms of parallelism, the following main advantages are achieved:

- **Maximization of Resource Throughput:** Instead of being limited to searching for independent instructions within a single thread (ILP), the processor can draw from multiple threads simultaneously to fill the available issue slots in every clock cycle.
- **Efficiency in Functional Unit Usage:** If one thread encounters a high-latency event, such as a stall due to memory access, instructions from other threads can utilize any remaining free functional units, preventing the hardware from sitting idle.
- **Optimization of Underlying Architecture:** The ideal exploitation of resources is limited only by the imbalance between requests and resource availability, allowing the architecture to stay as active as possible and reducing idle cycles.
- **Dynamic Adaptation:** The system is capable of adapting dynamically to the execution environment, allowing a single thread to utilize all resources if other threads are stalled or inactive.

By combining ILP and TLP through SMT, the highest possible saturation of the processor's computing units is achieved, overcoming the efficiency limits that would exist when using only one of the two techniques.

6.2 Which HW mechanisms of dynamic scheduled processors intrinsically support this approach?

The hardware mechanisms of dynamic scheduling processors that are already present and exploited to intrinsically support Simultaneous Multithreading (SMT) are those originally designed to maximize Instruction Level Parallelism (ILP).

Here is a summary of the "ready-made" components within the architecture:

- **Large Set of Registers:** Dynamic scheduling, especially with explicit register renaming, already utilizes a physical register file that is much larger than the number of registers defined by the ISA. This abundant space is fundamental for hosting the contexts of multiple threads simultaneously.
- **Register Renaming:** This mechanism, already used to eliminate name hazards (WAR and WAW), allows instructions from different threads to use the same logical register names without conflicting, by mapping them to distinct physical registers.
- **Out-of-Order Execution:** Since the hardware is already capable of reordering instructions to reduce stalls, the SMT approach simply needs to draw from the "ready" instructions of any available thread to fill the remaining free functional units.
- **ROB (Reorder Buffer):** The ROB is already present to manage speculative execution and ensure in-order commit. In SMT, it ensures that the results of each thread are finalized correctly, separating the moment of instruction completion from its official retirement into the processor state.

SMT does not require a hardware revolution; instead, it "inherits" these distributed control and buffering structures to intermix the execution flows of multiple threads.

6.3 Which modifications to a generic architecture of dynamically scheduled superscalar processors are required

To transform a generic dynamic scheduling architecture into one that supports Simultaneous Multithreading (SMT), specific hardware components must be added to handle multiple contexts simultaneously.

The main modifications to be added are:

- **Multiple PCs (Program Counters):** To manage multiple threads, the architecture must replicate the Program Counter (PC), assigning an independent one to each fetch unit. This allows the processor to fetch instructions from different execution flows within the same cycle.
- **Separation of the Reorder Buffer (ROB):** Since the ROB is responsible for ensuring in-order commit (ordered completion), it must be adapted in an SMT environment to handle each thread's flow separately. The separation or distinct management within the ROB ensures that a thread's instructions are finalized correctly without interfering with the speculative execution of other active threads.
- **Architectural Register State per thread:** each thread has its own architectural registers

While SMT leverages many existing out-of-order structures, the replication of the Instruction Fetch logic and the logical partitioning of the Retirement logic are the essential "glue" that allows multiple independent programs to share the same execution core.

7 MULTIPLE CHOICES

7.1 SIMD architectures can exploit significant data parallelism in a single operation

True, because SIMD stands for Single Instruction Multiple Data

7.2 Multithreading aims at speeding up the execution time of each individual thread

False ! Multithreading aims at speeding up the overall instruction execution throughput of the various threads, but it is not useful to speed up an individual thread execution. As a matter of fact, the execution time of each individual thread is actually worsened.

7.3 When using Explicit Register Renaming the ROB is no longer necessary but it can still be used to achieve precise exceptions in an easier way

The answer is **TRUE**.

Other strategies may be used but using the ROB still represents an "easy" solution

7.4 Mark the True answers to the following question , note that It might have multiple true answers

Which complications are introduced to MIPS pipelined processors by adding out-of-order execution and out-of-order completion

- Answer 1: WAR and WAW hazards
- Answer 2: RAW hazards
- Answer 3: Interrupt/Exception handling
- Answer 4: Control hazards
- Answer 5 : Instruction Reordering

A1 and A3 are TRUE.

Out-of-order execution introduces the possibility of WAR and WAW hazards, which do not exist in the five-stage MIPS in-order pipeline.

Out-of-order completion creates major complications in handling exceptions.

Dynamic scheduling with out-of-order completion must preserve exception behavior in the sense that exactly those exceptions that would arise if the program were executed in strict program order actually do arise.

8 VLIW Architecture has to have multiple Program Counters to load the necessary Multiple Data

FALSE!

A VLIW architecture is a SINGLE instruction architecture (the bundle is a SINGLE Very Long Instruction), therefore we need just one Program Counter.

A VLIW architecture is a SINGLE instruction architecture (the bundle is a SINGLE Very Long Instruction), so it works on a single control flow, therefore we need just one Program Counter.

Is the compiler that creates the bundle and then we fetch the Long instruction word as a single instruction

9 Explain the key idea of Dynamic Scheduling. Describe two dynamic-scheduling-based architectures and explain their main differences.

The **key idea behind dynamic scheduling** is to allow hardware to reorder instruction execution at runtime to reduce stalls caused by data dependencies that cannot be hidden by techniques like forwarding.

Instead of halting the entire pipeline when an instruction encounters a dependency, the processor allows subsequent instructions (behind the stall) to proceed if their operands are ready and hardware resources are available.

This approach enables out-of-order execution and completion, shifting the responsibility of managing parallelism from the compiler directly to the architecture.

The two main architectures based on dynamic scheduling are the Scoreboard and Tomasulo's Algorithm.

- **Scoreboard (CDC 6600) (4 stages architecture):**

The Scoreboard was the first example of dynamic scheduling and relies on centralized hazard management.

Operation: Every instruction passes through the Scoreboard, which determines when it is safe to read operands, begin execution, and write results. It divides the decode stage into two phases: Issue (in-order emission, where structural and WAW hazards are checked) and Read Operands (out-of-order reading, where the system waits for RAW resolution).

Hazard Management: If a functional unit is busy or if another active instruction has the same destination register (WAW), the instruction is stalled at the Issue stage. WAR hazards are managed by stalling the Write Result phase until previous instructions have finished reading the old values.

- **Tomasulo's Algorithm (3 stages architecture):** Tomasulo's algorithm improves upon the Scoreboard by distributing the control logic and operand buffers, called Reservation Stations, to the individual functional units.

Operation: Instructions are sent to Reservation Stations where they wait for their data. A fundamental feature is the Common Data Bus (CDB), a network that broadcasts results to all functional units, registers, and the reorder buffer simultaneously, allowing dependent instructions to "wake up."

Register Renaming: Tomasulo implements implicit register renaming. When an instruction is issued, the operand register names are replaced with actual values (if available) or with tags (pointers) to the Reservation Stations that will produce the result.

Notice: The presence of RSs is a key feature for a HW-based Loop Unrolling.

The fundamental differences between the two architectures include:

- **Control:** The Scoreboard uses a centralized control unit, while Tomasulo distributes control across the Reservation Stations.
- **Register Renaming:** The Scoreboard lacks renaming (except in explicit extended versions), so it must stall to avoid WAW and WAR conflicts. Tomasulo uses implicit renaming, eliminating WAR and WAW hazards at the source and allowing for greater fluidity.
- **Data Propagation:** In a Scoreboard, results pass through the registers. In Tomasulo, data is sent via the CDB, allowing instructions to receive data as soon as it is ready, bypassing the need to write to the register file first.
- **Stalls:** The Scoreboard stalls at Issue when a WAW is present; Tomasulo, thanks to renaming, continues issuing and only stalls if the Reservation Stations are full.

10 Increasing the number of stages in a pipeline, it is always improving the performance

FALSE!

Increasing the number of stages in a pipeline may lead to a higher number of overlapping situations but also to a much more complex control unit which may not improve, in the end, the performance of the architecture.

11 In a 5-stage pipeline MIPS architecture, it is possible to forward a data twice from stage-i to stage-j.

As, an example, given the following code:

```
I1 ADD F2, F4, F6
I2 ADD F4, F2, F26
I3 ADD F6, F2, F4
```

F2 will be forwarded from I1 to both I2 and I3 by using the EX → EX forwarding path?

NO!

Once data is moving from one stage to the following one, we cannot use any longer the previous stage to forward it!

12 Considering the pipeline with multiple floating-point functional units, in which the EX cycle is repeated as many times as needed to complete the operations, Which of the following statements are TRUE?

We need to know that the integer requires 1cc, the FP/Integer multiply requires 7 cc, the FP adder needs 4cc latency and the FP/Integer divider needs a lot of cc (more than multiply).

The statements are:

- Multiple instructions can be issued during the same clock cycle
- Instructions cannot complete in different order in which they were issued
- The number of registers write required in a cycle can be greater than 1
- Requires the introduction of additional pipeline registers

The TRUE statements are the third and the fourth.

Multiple operations cannot be active at the same time because the repetition represents a single stage (EX).

Furthermore, introducing the repetition, we also need to introduce new registers (between the repeated stages) and we need also to modify the connections between the ID stage and the first repetition of each FU.

Because of the fact that the different operations have varying running times, it can happen that sequences of FP operations try to write in the register write at the same time.

Moreover, the different running times does not ensure that the commit order of the instructions is the same as their issue order.

13 Which of the following statements about branch prediction are FALSE ?

- The behavior of each branch is always assumed independent from other branches
- Branch prediction can only happen at compile time
- Different techniques can be combined in order to improve prediction accuracy
- Profile - Driven is an example of a static branch predictor

The **FALSE** statements are the first and the second

Different techniques make different assumptions in order to make a prediction, for example an n-bit-BHT only considers the history of the branch to be predicted (with n regulating how much of the past history to consider).

An (m,n) Correlating Branch predictor on the other hand, takes into account the behavior of the last m branches.

Branch prediction can happen both at compile time (static) or during the execution (dynamic), the two approaches we briefly described above are examples of dynamic branch prediction techniques while.

Profile-Driven or Delayed Branch are static predictors.

Static predictions may be used to assist dynamic predictors.

14 The “Branch Always Taken” technique is not applicable to the five-stage MIPS pipeline! Is this statement true or false?

It is **true**, in this technique as soon as the branch is decoded, and the target address is computed, we assume the branch to be taken and begin fetching and executing at the target.

Because in the five-stage MIPS pipeline we don't know the target address any earlier than we know the branch outcome, there is no advantage to this approach.

15 What is the reason a 1 bit-BHT is usually outperformed by a 2 bit-BHT?

- A 2-bit BHT leverages more information about the history of a branch
- The use of 2 bits allows to index a larger number of branch instructions
- The difference in performance is not significant
- The use of 2 bits improves the accuracy of the associated BTB

The **TRUE** statement is only the first.

A 2-bit BHT needs two mispredictions in order to change prediction.

This means it also leverages more of the past history of the branch in order to make its predictions.

The improvement is clear for example in the case of two nested loops where the mispredictions on the inner loop are halved with respect to the 1 bit-BHT

16 In the context of branch hazards, which of the following assertions are FALSE?

- The Early Evaluation of PC behaves in the same way regardless of the preceding instructions
- The Early Evaluation of PC computes branch target address during the EX phase
- With only forwarding and stalling, it requires two clock cycles until the branch decision is taken
- The Early Evaluation of PC could need stalls in some particular cases

FALSE statements are the first and the second.

1. If a comparison register is a destination of preceding ALU instruction or 2nd preceding load instruction a further stall is needed, two in the case of an immediately preceding load instruction.
2. The target address is computed during the ID phase.
3. The result is available in the register after the EX phase of the branch instruction.
4. For what stated before.

17 The following 5 questions are True/False

1. A scalar architecture can let instructions from different threads to be executed at the same clock cycle.

FALSE: a scalar architecture is a single issue architecture

2. The ARM big.LITTLE architecture, considering the fact that is a multicore architecture where the cores are sharing the same ISA, cannot be considered a heterogeneous architecture.

FALSE: The (at least) two cores are different, therefore, it's an heterogeneous architecture even if they are sharing the same ISA

3. A physically distributed memory can be used to implement a shared-memory architecture.

TRUE: Both the centralized memory and the distributed memory models can be used to implement a shared-memory architecture

4. A VLIW Architecture has to have multiple Program Counters to load the necessary Multiple Data.

FALSE: A VLIW architecture is a SINGLE instruction architecture (the bundle is a SINGLE Very Long Instruction), therefore we need just one Program Counter

5. Increasing the number of stages in a pipeline, it is always improving the performance

FALSE: Increasing the number of stages in a pipeline may lead to a high number of overlapping situations but also to a much more complex control unit which may not improve, in the end, the performance of the architecture.

18 Explain the difference among Dependencies and Hazards.

The fundamental difference between dependencies and hazards lies in their nature: dependencies are a property of the program, while hazards are a property of the hardware pipeline. Here is a detailed breakdown of the distinctions:

1. **Dependencies (Software Property):** Dependencies represent a logical constraint between two instructions that prevents them from being executed in parallel or requires a specific order to ensure the correctness of the result.

There are three main types:

- **Data Dependencies (True Dependencies):** Occur when an instruction produces a result used by a subsequent instruction.
- **Name Dependencies:** Occur when two instructions use the same register or memory location without an actual data flow between them (divided into anti-dependencies and output dependencies).
- **Control Dependencies:** These determine the order of instructions relative to a conditional branch.

2. **Hazards (Hardware Property):** A hazard occurs when the pipeline configuration prevents an instruction from executing in its designated clock cycle. In practice, a hazard is the physical manifestation of a dependency within a pipeline.

They are created when dependent instructions are so close together that the overlap caused by pipelining would change the order of access to operands.

They reduce performance compared to the ideal pipeline speedup.

They are classified as **structural** (resource conflicts), **data** (RAW, WAR, WAW), and **control** (related to branches)

Not all dependencies necessarily turn into hazards. For example, if two dependent instructions are sufficiently far apart in the program sequence, the pipeline might resolve the dependency naturally before a conflict occurs. Furthermore, hardware techniques such as forwarding (bypassing) can eliminate a hazard while the logical dependency remains in the code.

In short:

- **Dependency:** "Instruction B needs the result of Instruction A" (The theoretical limit of the code).
- **Hazard:** "Instruction B cannot execute right now because the pipeline hasn't finished Instruction A yet" (The physical limit of the processor).

19 Explain the main differences between loop unrolling and software pipelining?

Software pipelining can be thought of as "symbolic" loop unrolling: the main difference is that loop unrolling needs to add some startup code before the loop as well as finish up code at the end.

On the other hand, loop unrolling needs to replicate the entire code of the loop.

For what concern the register pressure, the amount of registers used are not related to the type of scheduling but only to the architecture used.

Due to the fact that loop unrolling at each iteration needs to repeat the computation, it needs to pay the cost of overhead and wind-down more times than software pipelining.

For what concerns the power consumption, because of the fact that they are both static scheduling techniques, the power consumption is lower wrt dynamic scheduling techniques.

Hence, power consumption does not depend on the type of scheduling technique used.

20 The following 3 questions are True/False.

1. A superscalar architecture can let instructions from different threads to be executed at the same clock cycle

TRUE: A superscalar architecture has several program counters to let instructions from different threads to be executed at the same clock cycle.

This technique is also known as **simultaneous multithreading**

2. The ARM big.LITTLE architecture, considering the fact that is a multicore architecture where the cores are sharing the same ISA, cannot be considered a MIMD architecture but a SIMD one.

FALSE: A multicore architecture with different program counters, even if the cores are sharing the same ISA, is a MIMD architecture

3. A physically centralized memory can be used to implement a shared-memory architecture.

TRUE: Both the centralized memory and the distributed memory models can be used to implement a shared-memory architecture

21 Explain the difference among fine-grained multi threading, coarse-grained multi threading and simultaneous multi threading.

The sources describe three main approaches for implementing **Thread-Level Parallelism (TLP) within a processor**: Fine-grained Multithreading, Coarse-grained Multithreading, and Simultaneous Multithreading (SMT)

1. Fine-grained Multithreading

- **Operation:** This mechanism switches between threads after every instruction (or clock cycle), alternating the execution of various flows.
- **Hardware Requirements:** The CPU must be capable of switching threads every clock cycle; this requires duplicating the architectural state of the thread (such as the Program Counter and registers), while functional units remain shared.
- **Advantages:** It can hide both short and long stalls, as instructions from other threads can be executed when one thread is blocked.
- **Disadvantages:** The main drawback is that it slows down the execution of an individual thread, as a ready thread must still wait for its turn among the others.

2. Coarse-grained Multithreading

- **Operation:** Thread switching occurs only in the presence of significant and prolonged stalls, such as a Level 2 (L2) cache miss.
- **Context Management:** In this case as well, each thread possesses its own architectural state; however, switching threads involves an overhead of several clock cycles because the pipeline must be emptied (flushed) before executing instructions from the new thread.
- **Advantages:** Under normal conditions, individual thread performance is not slowed down, as instructions from other flows are only issued when a costly stall occurs.
- **Disadvantages:** This approach does not reduce the throughput loss caused by short stalls, as the processor does not perform a thread switch until the stall is completed.

3. Simultaneous Multithreading (SMT)

- **Operation:** SMT combines Instruction-Level Parallelism (ILP) and Thread-Level Parallelism (TLP), allowing the simultaneous execution of instructions from different threads within the same clock cycle.
- **Efficiency:** The system adapts dynamically, allowing a single thread to utilize all functional units if other threads encounter long-latency events. Ideally, resource usage is limited only by the imbalance between requests and hardware availability.
- **Implementation:** To enable this approach, the processor replicates the Program Counter and the architectural state for each thread, while the fetch, decode, issue, and execution units are shared and can issue instructions belonging to different threads in the same cycle.

22 Explain the main differences between a VLIW architecture and a superscalar architecture, in terms of instructions scheduling, hardware complexity, and code size.

VLIW (Very Long Instruction Word) and Superscalar architectures represent two different approaches to breaking the CPI = 1 barrier by exploiting Instruction-Level Parallelism (ILP). While both aim to execute multiple operations per clock cycle, they differ radically in how they manage resources and conflicts.

Here are the main differences analyzed according to the requested criteria:

1. Instruction Scheduling

- **VLIW (Static Scheduling):** The responsibility for identifying parallelism and managing dependencies falls entirely on the compiler. The compiler groups a fixed number of independent operations (ranging from 4 to 16) into a single "very long instruction" (bundle or template). The code provided to the hardware must be dependency-free, as the processor does not perform runtime checks.
- **Superscalar (Dynamic Scheduling):** The search for parallelism occurs at runtime via hardware. The processor utilizes an "issue window" within which it analyzes dependencies (RAW, WAR, WAW) and decides which instructions can be issued and executed in parallel.

2. Hardware Complexity

- **VLIW:** Features reduced hardware complexity regarding the control unit, as it does not need to manage conflicts or reorder instructions at runtime. However, it requires a high number of registers and high bandwidth between the cache and the fetch unit to load the large instruction bundles.
- **Superscalar:** The hardware is extremely complex. It must integrate logic for register renaming, reservation stations, and reorder buffers (ROB). The need to perform thousands of comparisons per clock cycle to detect hazards makes these architectures expensive in terms of silicon area and less energy-efficient.

3. Code Size

- **VLIW:** Tends to have a very large code size (code explosion). This happens because if the compiler cannot fill all the slots of a bundle with useful operations (due to a lack of parallelism in the original code), it must insert explicit NOP (No Operation) instructions, wasting memory space.
- **Superscalar:** The code is more compact and efficient. Since the hardware dynamically decides what to execute in parallel starting from a standard instruction stream, it is not necessary to fill the binary file with static NOPs to occupy unused execution slots.

23 In real superscalar processors, some architectural limits do not allow fully exploitation of the ILP present in the program. Specify these architectural limits and specify how they impact on the potential ILP

In real **superscalar processors**, the exploitation of **Instruction-Level Parallelism (ILP)** is restricted by several architectural constraints that prevent achieving the performance of an "ideal machine."

Below are the primary architectural limits and their impact on potential ILP:

1. Instruction Window Size

The instruction window defines how many instructions the processor can "see" simultaneously to search for independent operations to execute.

Impact: A limited window narrows the scope of the search for parallelism. Furthermore, increasing the window size drastically increases hardware complexity, as the number of comparisons required to detect RAW (Read After Write) dependencies grows quadratically ($n^2 - n$).

2. Issue Width

This refers to the maximum number of instructions that can be sent to functional units in a single clock cycle.

Impact: While an ideal machine could issue an infinite number of instructions, real processors are limited (usually between 3 and 6 instructions per cycle). Doubling the issue rate would require an unsustainable increase in register ports, memory accesses, and renaming logic.

3. Physical Resource Limits

These include the number of Functional Units (FUs), communication buses, and register file ports.

Impact: Even if independent instructions are present in the window, they cannot execute if hardware resources (such as arithmetic units or data buses) are already saturated. For example, many processors do not support more than two memory references per cycle.

4. Branch and Jump Prediction

Unlike an ideal model with perfect prediction, real processors make mistakes.

Impact: Prediction errors cause a pipeline flush, wasting clock cycles and drastically reducing exploitable ILP beyond basic blocks.

5. Memory Alias Analysis

This is the hardware's ability to determine if two different memory references point to the same location.

Impact: Uncertainty regarding memory dependencies prevents the processor from freely reordering Load and Store operations, limiting potential parallelism in code sections that make heavy use of memory.

6. Register Renaming

This depends on the limited number of physical registers available compared to those defined by the ISA (Instruction Set Architecture).

Impact: If physical registers are exhausted, the processor must stall the issuance of new instructions—even if they have no true data dependencies—to avoid name conflicts (WAR and WAW).

7. Heat Dissipation and Power Consumption

The increased complexity required to extract more ILP leads to power consumption that grows faster than the resulting performance gains.

Impact: This represents an insurmountable physical limit (the so-called "Power Wall"), making techniques to increase ILP energy-inefficient beyond a certain point.

24 Explain what Amdahl's Law is, what it is used for, and its implications

Amdahl's Law is a mathematical model used in computer architecture to establish a standard process for measuring the performance improvement (speedup) resulting from a specific optimization.

In essence, it quantifies how much an enhancement of a single part of a system affects the total execution time of a task. What is Amdahl's Law? The law is based on the concept that an enhancement E accelerates a fraction F (Fraction enhanced) of a task by a factor S (Speedup enhanced), while the remaining part of the task remains unchanged.

The fundamental formula for calculating the overall speedup is:

$$Speedup_{overall} = \frac{1}{(1 - F_{enhanced}) + \frac{F_{enhanced}}{S_{enhanced}}}$$

Amdahl's Law serves fundamental purposes in system design and analysis:

- **Performance Evaluation:** It allows for the measurement of the real impact of a hardware or software upgrade before it is implemented.
- **Design Space Exploration:** It helps designers determine if investing resources in a specific upgrade is worthwhile by comparing the performance gain against the necessary costs.
- **Bottleneck Identification:** It highlights that there is no point in infinitely boosting a single component if the system is limited by another non-improvable fraction.

The implications of this law are crucial for computer architecture:

- **Make the Common Case Fast:** In design, priority must be given to optimizing operations that are executed most frequently, as they have a greater impact on the fraction F of the total time.
- **Maximum Speedup Limit:** There is an insurmountable theoretical limit to performance improvement. Even if you made the enhanced fraction infinitely fast, the maximum speedup would still be limited by the unenhanced part:

$$Speedup_{max} = \frac{1}{1 - F}$$

- **Cost-Performance Ratio:** A massive increase in a component's speed (e.g., a CPU that is 10x faster) may result in a minimal benefit for the user (e.g., only 1.56x faster) if that component is used for only a small fraction of the time, making the investment economically inefficient.
- **Impact of External Bottlenecks:** If a system is "I/O bound" (limited by input/output), drastically increasing the CPU's computing power will be of little use, as the fraction of time spent waiting for I/O will dominate the total time.

25 What is the purpose of PC Early Evaluation and how do you apply it?

Early Evaluation of the PC is a technique used to manage control hazards caused by conditional branch instructions within a pipeline.

What is it used for?

The primary goal is to improve performance by reducing the so-called "branch penalty." In a standard MIPS pipeline, the decision of whether to take a branch and the calculation of the Branch Target Address (BTA) are typically completed at the end of the MEM stage (4th stage).

This delay forces the processor to:

- Suspend execution (insert stalls) until the result is known.
- Or "flush" incorrectly loaded instructions if the branch is taken, wasting up to 3 clock cycles.

Early Evaluation moves these operations to the initial phases of the pipeline to minimize the number of wasted cycles.

To implement this technique, the architecture must be modified by adding specific hardware resources to advance decisions to the ID (Instruction Decode) stage. The main modifications include:

- **Moving the Comparison Logic:** A hardware comparator is added to the ID stage to determine the branch outcome (e.g., if registers are equal or not) as soon as the registers are read.
- **Early BTA Calculation:** The dedicated branch adder unit is moved to the ID stage to calculate the target address (by adding the offset to the current PC) immediately after decoding.
- **Fast PC Update:** The result of the comparison is used to drive a multiplexer in the IF (Instruction Fetch) stage, allowing the system to choose between PC+4 and the BTA as early as the very next cycle.

Results and Implications:

- **Penalty Reduction:** Thanks to this technique, the penalty for each branch is reduced to just a single clock cycle stall (or the flushing of only one instruction), instead of the original three cycles.
- **New Data Hazards:** A side effect is that by advancing the register read for the branch, new data hazards may arise. If the instruction immediately preceding the branch calculates a value needed by the branch itself, an additional stall must be inserted because forwarding might not be sufficient to provide the data in time for the ID stage.

26 Explain the static branch prediction techniques

Static branch prediction techniques are methods used to mitigate performance loss caused by control hazards, with decisions fixed at compile-time or design-time. These techniques are effective when branch behavior is highly predictable.

Core techniques:

- **Branch Always Not Taken:** In this scheme, the hardware assumes the branch is never executed, continuing to fetch instructions sequentially. If the prediction is correct, performance remains optimal. If the branch must be taken, the incorrectly loaded instruction is transformed into a NOP (flush), and execution resumes from the branch target address, incurring a one-clock-cycle penalty.
- **Branch Always Taken:** It is assumed that the branch will always be executed; as soon as the instruction is decoded and the target address is calculated, the processor begins fetching instructions from that address. In pipelines like MIPS, this technique offers no real advantage because the target address is often not known before the branch outcome itself is determined.

- **Backward Taken Forward Not Taken (BTFTNT):** Prediction is based on the direction of the branch: backward branches, typical of loop closures, are predicted as "taken," while forward branches are predicted as "not taken."
- **Profile-Driven Prediction:** This technique uses information gathered during previous executions of the program to determine the probability of a branch, sometimes utilizing compiler hints.
- **Delayed Branch:** The compiler attempts to insert an independent instruction into the so-called branch delay slot that follows the branch instruction. This instruction is always executed, regardless of whether the branch is taken or not.

The compiler can fill this slot in three ways:

- **From Before:** By moving an independent instruction that was located before the branch.
- **From Target:** By copying the instruction from the branch target address (useful for loops).
- **From Fall-Through:** By choosing the instruction that would sequentially follow the branch (useful for forward branches).

27 Why superscalar processors uses dynamic techniques?

Superscalar processors utilize dynamic techniques primarily to reduce pipeline stalls caused by data dependencies that the compiler cannot hide or predict. By allowing the hardware to reorder instruction execution at runtime, the processor can advance instructions located "behind" a stall, provided their operands are ready.

Key reasons for using these techniques include:

- **Maximizing ILP:** The fundamental goal is to extract as much Instruction-Level Parallelism (ILP) as possible to overcome the ideal limit of one instruction completed per clock cycle (aiming for a CPI $\downarrow$ 1).
- **Managing Runtime Uncertainties:** Dynamic techniques allow for the identification of parallelism even when dependencies are unknown at compile time, such as in the case of memory accesses with variable addresses.
- **Resolving Name Hazards:** The use of dynamic techniques like register renaming is essential for eliminating WAR (Write After Read) and WAW (Write After Write) hazards, which occur frequently when instructions are executed out of order.
- **Overcoming Basic Blocks:** Thanks to hardware speculation and dynamic branch prediction, processors can execute instructions beyond the boundaries of individual basic blocks, acting even before the outcome of a branch is resolved.
- **Adaptability:** These architectures allow code written for one machine to run efficiently on different models, as the hardware itself adapts the execution to the available resources.

28 Explain the different types of MIMD architectures (shared/distributed memory, shared/private address spaces)

The MIMD architecture is the most common type of parallel computer, in which each processor executes a different instruction stream operating on independent data streams. These architectures are extremely flexible, capable of serving as high-performance machines for a single user or as multiprogrammed systems running many tasks simultaneously.

There are two main criteria for classifying MIMD architectures: the **address space model (logical)** and the **physical memory organization**. These concepts are orthogonal: for example, it is possible to have a shared address space implemented on physically distributed memory.

1. Address Space Models (Logical View)

This distinction concerns the view of the memory from the programmer's perspective and how processes communicate with each other:

- **Shared Memory (SM):** Any processor can reference any memory location, allowing for implicit communication through shared variables.
- **Private/Distributed Memory (MP):** Each processor has its own memory space. Communication must occur explicitly through Message Passing (sending and receiving messages).

2. Physical Memory Organization

This distinction concerns the hardware arrangement of the memory modules:

- **Centralized Shared Memory (UMA – Uniform Memory Access):** Memory is unified or centralized, accessible by all processors with uniform latency. This is typical of SMP (Symmetric Multiprocessors) systems.
- **Distributed Memory (NUMA – Non-Uniform Memory Access):** Memory is divided among different nodes; accessing local memory is faster than accessing remote memory, requiring high-bandwidth interconnections.

The choice between SM and MP architectures depends on trade-offs between ease of programming and scalability:

- **Shared Memory:** Offers low overhead but scaling to large systems is difficult and requires complex cache coherence management.
- **Message Passing:** Easier to scale for massively parallel systems and provides greater control over data placement, but introduces software overhead and requires the programmer to manage communication explicitly.

29 GPUs and Heterogenous Computing Systems

The evolution of architectures toward heterogeneous systems stems from the need to overcome the power and bandwidth limitations of traditional processors by combining specialized computing units to maximize performance.

The fundamental distinction lies in the execution model:

- **CPU (Central Processing Unit):** Designed for the serial execution of complex programs. It possesses few execution units, which are optimized for high clock frequencies and managing conditional branches.
- **GPU (Graphics Processing Unit):** A specialized processor for massively parallel operations. While a CPU has short pipelines (10–20 stages), GPUs feature extremely deep pipelines (up to several thousand stages) and much faster memory interfaces to handle massive data streams.

The GPU's workload is divided into logical stages that transform geometric data into images:

- **Host Interface:** Receives commands and data from the CPU.
- **Vertex Processing:** Transforms vertices from "object space" to "screen space."
- **Triangle Setup:** Converts geometric data into raster information (pixels).
- **Pixel (Fragment) Processing:** Calculates the final color of each pixel, applying textures and mathematical operations. This stage is often the computational bottleneck.
- **Memory Interface:** Writes the final results into the framebuffer. Modern GPUs allow programmers to customize these stages via dedicated programs called shaders.

GPUs are extremely efficient for:

- Parallel Floating Point processing.
- SIMD (Single Instruction Multiple Data) operations.
- High computation-to-memory access ratios. They are less efficient in tasks with heavy branching or random memory access.

In a heterogeneous system, different processors cooperate to solve a problem. A classic example is the Xbox 360, where the Xenon CPU handles logic and physics while the GPU manages graphics.

- **Communication:** Occurs via a Command Buffer. The CPU writes commands, and the GPU reads and executes them asynchronously.
- **Synchronization:** If the CPU attempts to modify data still in use by the GPU, the system must use semaphores (locks), which can cause CPU stalls ("spinning"). To avoid this, data renaming is used, allocating new memory blocks instead of overwriting old ones.
- **Limitations:** Data readback (from GPU to CPU) is strongly discouraged because it destroys parallelism, forcing the CPU to wait for the entire GPU pipeline to flush.

30 Static vs Dynamic branch prediction

The fundamental difference between static and dynamic branch prediction lies in when the decision regarding the branch direction is made and the system's ability to adapt to the program's behavior during execution.

1. Static Branch Prediction

In static prediction, the actions taken for a branch are fixed at compile-time or design-time and never change during execution. This approach is used when branch behavior is considered highly predictable before the program even runs.

The main techniques described are:

- **Branch Always Not Taken:** The hardware assumes the branch is never taken, continuing with sequential execution. If the prediction is correct, there is no penalty; if incorrect, the loaded instruction is discarded (flushed), causing a one-clock-cycle penalty.
- **Branch Always Taken:** It is assumed that the branch is always taken; however, in the MIPS architecture, this offers no advantage because the target address is not known before the branch outcome itself is determined.
- **BTFNT (Backward Taken Forward Not Taken):** This relies on the direction of the branch: backward branches (typical of loops) are predicted as taken, while forward branches are predicted as not taken.
- **Delayed Branch:** The compiler moves an independent instruction into a "delay slot" after the branch; this instruction is always executed regardless of whether the branch is taken or not, effectively eliminating the stall penalty.

2. Dynamic Branch Prediction

Dynamic prediction uses dedicated hardware to predict the outcome at runtime, basing its guess on the past behavior of the branch. Unlike static prediction, the decision can change if the branch behavior shifts during execution.

It relies on two main components:

- **Branch History Table (BHT):** A table indexed by the lower bits of the branch address containing "hints."
 - **1-bit BHT:** Simply stores whether the branch was taken last time; it changes its prediction upon the first error.
 - **2-bit BHT:** More resilient, requiring two consecutive errors before changing the prediction (highly useful in nested loops to avoid errors at the start and end of the loop).
- **Branch Target Buffer (BTB):** A cache that stores the branch target address, allowing the processor to begin fetching instructions from the new destination even before the address is calculated by the ALU.

Sintesi delle differenze principali

Caratteristica	Predizione Statica	Predizione Dinamica
Decisione presa da	Compilatore / Progettista 1	Hardware a runtime 7
Adattabilità	Rigida (non cambia mai) 1	Flessibile (si adatta alla storia) 7
Risorse Hardware	Minime o nulle	Elevate (tabelle BHT, buffer BTB) 8 14
Efficacia	Alta nei loop semplici	Molto alta in programmi complessi 16 17

Figure 1

31 Is SIMD just an architecture or is it an extension of instruction set?

SIMD (Single Instruction, Multiple Data) can be both, but it depends on the context:

- **As an architectural concept:** SIMD is a parallel computing paradigm where a single instruction operates on multiple data elements simultaneously. This concept can exist at the architectural level in CPUs, GPUs, or specialized processors. For example, vector processors in supercomputers are designed around SIMD principles.
- **As an instruction set extension:** In modern CPUs (like x86), SIMD is often implemented as extensions to the base instruction set, it appears as instruction set extensions that expose SIMD functionality to software.

32 Talk about Cell and Xenon

These two processors represent different approaches to multiprocessor system design, created to meet the computational demands of video game consoles (PlayStation 3 and Xbox 360) and high-performance computing.

1. The Cell Processor

The Cell is defined as a **heterogeneous on-chip multiprocessor**.

Its architecture does not consist of identical cores; instead, it combines different units to maximize efficiency:

- **Structure:** It is composed of one main 64-bit Power core and eight specialized co-processors called SPU (Synergistic Processor Units).
- **SPU Architecture:** Each SPU is based on a new SIMD (Single Instruction Multiple Data) architecture, designed to handle specific, computationally intensive tasks.
- **Objective:** The choice of heterogeneity allows a general-purpose core (the Power core) to manage the operating system and complex logic, while the SPUs handle the parallel processing of large data streams. This makes it extremely powerful for vector and multimedia calculations.

2. The Xenon Processor

The Xenon, used in the Xbox 360, follows a more traditional approach compared to the Cell, based on a homogeneous architecture.

- **Structure:** It is a processor composed of a single core replicated three times (tri-core).
- **Core Characteristics:** Each core is a superscalar processor with two issue windows and includes a SIMD extension to manage multimedia and graphical workloads.
- **Role in the System:** Within the console, the Xenon manages tasks such as physics, game logic, and user interaction, working asynchronously with a dedicated GPU for graphical rendering.

Key Differences and Heterogeneous Cooperation

- **Heterogeneity vs. Homogeneity:** The Cell is a heterogeneous system (different cores working together), while the Xenon is homogeneous (replicas of the same core).
- **Workload Management:** While the Cell attempts to perform most of the "heavy lifting" internally via the SPUs, the Xenon is designed to operate within a broader heterogeneous system. In this setup, the CPU prepares simplified data (such as high-level geometric descriptions) which is then translated and rendered by the GPU, thereby overcoming memory bandwidth limitations.

33 In MIMD architectures with a physically centralized memory organization, the address space is shared (True/False)

The answer to your question is **False**.

In MIMD architectures, the (logical) address space model and the physical memory organization are orthogonal concepts. This means that a specific physical organization does not necessarily dictate a single logical model."

34 Which type of hazards can be caused by the name dependencies?

- Answer 1: WAR
- Answer 2: RAR
- Answer 3: WAW
- Answer 4: RAW

Name dependencies occur when two instructions use the same register or memory location (the "name"), but there is no actual data flow between them. According to the sources, these dependencies can cause the following types of hazards:

- Answer 1: WAR (Write After Read): This hazard corresponds to an anti-dependence. It occurs when a subsequent instruction attempts to write to a register or memory location that a previous instruction has not yet read; the order must be preserved to prevent the first instruction from reading the wrong value.
- Answer 3: WAW (Write After Write): This hazard corresponds to an output dependence. It occurs when two instructions write to the same register or location; the order must be maintained so that the final stored value is that of the instruction that appears last in the program.

In contrast, Answer 4 (RAW) is caused by a true data dependence, where an instruction actually depends on the value produced by a previous one

35 Statically-scheduled superscalar processors are the current state-of-the-art for general purposes processors; some implementations include MIPS and Intel Core i processors (True/False)

FALSE: The state of the art for high-performance general-purpose processors is not static scheduling, but dynamic scheduling (out-of-order execution).

36 Theoretically, an unlimited number of issues will be the best option to achieve the highest performance (True/False)

FALSE: Performance does not scale linearly with the issue rate due to intrinsic conflicts within the code and the physical and power limits of the hardware.

37 Name dependent instructions (Name dependence occurs when 2 instructions use the same register or memory location) can execute simultaneously but their relative order of execution cannot be changed. (True/False + Effectively support your answer)

FALSE: If two instructions are dependent, they cannot be executed in parallel (therefore not simultaneously). They must be executed in order or can only be partially overlapped

38 It is not possible to have heterogeneous multicore architectures sharing the same ISA (True/False + Effectively support your answer)

FALSE: Heterogeneity in a multicore architecture refers to the presence of cores with different microarchitectural characteristics or hardware specializations to handle specific tasks, but this does not prevent them from implementing the same Instruction Set Architecture (ISA)